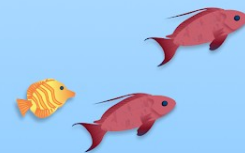


Wakacyjne Wyzwanie

Z  KN Solvro

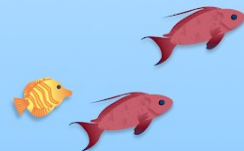


Wykład 5

Kontenerki część 2

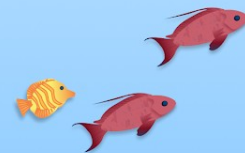


Co w planie dzisiaj?



- Szybkie kwestie organizacyjne
- Komunikacja między kontenerami
- Docker Compose
 - czyli jak łatwo stawiać grupy kontenerów
- DNS
- HTTPS



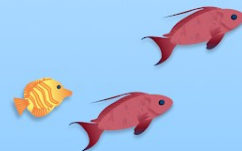


Rozdział 0

Kwestie organizacyjne



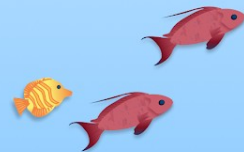
Hackaton



- Hackaton już za tydzień
 - Link do zapisów powinniście mieć na mailu
 - Czas na zapisy do poniedziałku
- Będzie prowadzony przez 1 lub 2 osoby z zespołu DevOps KN Solvro



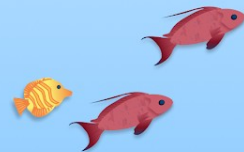
Co na hackatonie?



- Na ścieżce DevOps hackaton zastępuje projekt
 - Podzielicie się na grupy (sieci)
 - Będziecie stawiać usługi sieciowe wewnątrz sieci
 - Zrobicie routing między sieciami
- Koniecznie przygotujcie laptop z linuxem
 - Zalecana obecność portu RJ45
- Jeżeli macie nadal listy do zrobienia – jeżeli wykorzystacie techniki z list na hackatonie, zgłóście to prowadzącym – możecie w ten sposób zaliczyć listę

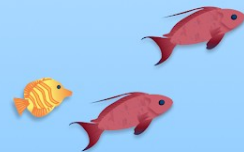


Co z wykładem za tydzień?



- Hackaton jest w tym samym czasie, co planowo miał być wykład
 - Więc przesuniemy wykład na czwartek



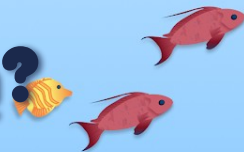


Rozdział 1

Komunikacja między kontenerami



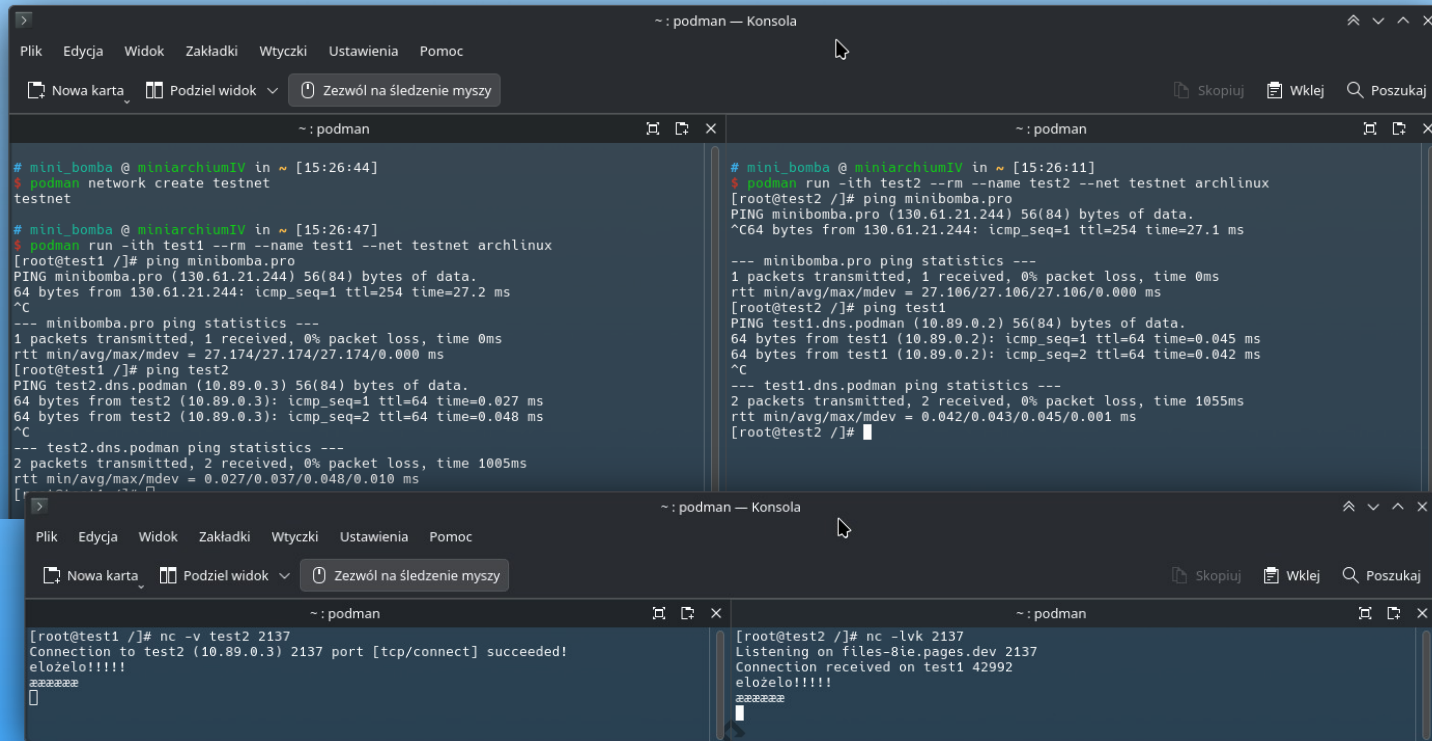
Jak kontenery komunikują się ze sobą?



- **Najczęściej przez wirtualne sieci**
 - Domyślne sieci dockerowe/podmanowe to są po prostu urządzenia bridge
 - + wewnętrzny DNS, który zna nazwy kontenerów
 - Sposób działania jest podobny do tego, co robiliśmy na liście 1
- **W teorii mogą też komunikować się przez system plików**
 - pliki/nazwane potoki/gniazda unixowe
 - Nie zalecałbym tego przy uruchamianiu kontenerów w klastrach



Przykład



The image displays four terminal windows from the Podman application, illustrating the setup and testing of a network. The windows are arranged in a 2x2 grid.

Top Left Window: Shows the creation of a testnet and the execution of a podman run command to create a test1 container.

```
# mini_bomba @ miniarchliumIV in ~ [15:26:44]
$ podman network create testnet
testnet

# mini_bomba @ miniarchliumIV in ~ [15:26:47]
$ podman run -ith test1 --rm --name test1 --net testnet archlinux
[root@test1 /]# ping minibomba.pro
PING minibomba.pro (130.61.21.244) 56(84) bytes of data.
64 bytes from 130.61.21.244: icmp_seq=1 ttl=254 time=27.2 ms
^C
--- minibomba.pro ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 27.174/27.174/27.174/0.000 ms
[root@test1 /]# ping test2
PING test2.dns.podman (10.89.0.3) 56(84) bytes of data.
64 bytes from test2 (10.89.0.3): icmp_seq=1 ttl=64 time=0.027 ms
64 bytes from test2 (10.89.0.3): icmp_seq=2 ttl=64 time=0.048 ms
^C
--- test2.dns.podman ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1005ms
rtt min/avg/max/mdev = 0.027/0.037/0.048/0.010 ms
```

Top Right Window: Shows the execution of a podman run command to create a test2 container and the execution of ping commands to test connectivity between test1 and test2.

```
# mini_bomba @ miniarchliumIV in ~ [15:26:11]
$ podman run -ith test2 --rm --name test2 --net testnet archlinux
[root@test2 /]# ping minibomba.pro
PING minibomba.pro (130.61.21.244) 56(84) bytes of data.
^C64 bytes from 130.61.21.244: icmp_seq=1 ttl=254 time=27.1 ms
--- minibomba.pro ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 27.106/27.106/27.106/0.000 ms
[root@test2 /]# ping test1
PING test1.dns.podman (10.89.0.2) 56(84) bytes of data.
64 bytes from test1 (10.89.0.2): icmp_seq=1 ttl=64 time=0.045 ms
64 bytes from test1 (10.89.0.2): icmp_seq=2 ttl=64 time=0.042 ms
^C
--- test1.dns.podman ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1055ms
rtt min/avg/max/mdev = 0.042/0.043/0.045/0.001 ms
[root@test2 /]#
```

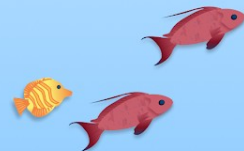
Bottom Left Window: Shows the execution of a netcat listener on test1 and the execution of a netcat client on test2.

```
[root@test1 /]# nc -v test2 2137
Connection to test2 (10.89.0.3) 2137 port [tcp/connect] succeeded!
elozele!!!!
=====
^C
```

Bottom Right Window: Shows the execution of a netcat listener on test2 and the execution of a netcat client on test1.

```
[root@test2 /]# nc -lvk 2137
Listening on files-8ie.pages.dev 2137
Connection received on test1 42992
elozele!!!!
=====
^C
```

Kontenery we wielu sieciach



- **Kontenery można dodać do wielu sieci naraz**
 - Można w ten sposób przy aplikacjach z wieloma serwisami ograniczyć ruch między serwisami, które nie powinny się ze sobą komunikować
 - Używane też przez kontenery z reverse proxy, by miały dostęp do kontenerów z innych sieci
- **Można kontenery też dynamicznie dodawać/usuwać z sieci**
 - Przydatne dla kontenerów z reverse proxy



Przykład

```
net1

# mini_bomba @ miniarchiumIV in ~ [15:44:45]
$ podman network create net2 --ipv6
net2

# mini_bomba @ miniarchiumIV in ~ [15:44:49]
$ podman network create net3 --ipv6
net3

# mini_bomba @ miniarchiumIV in ~ [15:44:58]
$ podman run -ith cont1 --name cont1 --rm --net net1 arch-with-nc
[root@cont1 /]# ping cont2
PING cont2 (fde2:d4e8:4db8:c1d1::3) 56 data bytes
64 bytes from cont2 (fde2:d4e8:4db8:c1d1::3): icmp_seq=1 ttl=64 time=0.049 ms
^C
--- cont2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.049/0.049/0.049/0.000 ms
[root@cont1 /]# ping cont3
ping: cont3: Name or service not known
[root@cont1 /]# ping cont4
ping: cont4: Name or service not known
[root@cont1 /]#

~: podman

# mini_bomba @ miniarchiumIV in ~ [15:40:11] C:
$ podman run -ith public-service --rm --name public-service --net podman --net intnet arch-wi
th-nc
[root@public-service /]# ping minibomba.pro
ping: minibomba.pro: Name or service not known
[root@public-service /]#
exit

# mini_bomba @ miniarchiumIV in ~ [15:46:19] C:
$ podman run -ith cont4 --name cont4 --rm --net net3 arch-with-nc
[root@cont4 /]# ping cont1
ping: cont1: Name or service not known
[root@cont4 /]# ping cont2
ping: cont2: Name or service not known
[root@cont4 /]# ping cont3
PING cont3 (fdd7:697e:b1f6:8a28::3) 56 data bytes
64 bytes from cont3 (fdd7:697e:b1f6:8a28::3): icmp_seq=1 ttl=64 time=0.028 ms
64 bytes from cont3 (fdd7:697e:b1f6:8a28::3): icmp_seq=2 ttl=64 time=0.044 ms
64 bytes from cont3 (fdd7:697e:b1f6:8a28::3): icmp_seq=3 ttl=64 time=0.042 ms
^C
--- cont3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2072ms
rtt min/avg/max/mdev = 0.028/0.038/0.044/0.007 ms
[root@cont4 /]#

~: podman

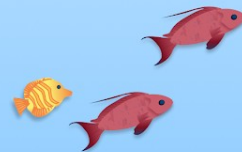
# mini_bomba @ miniarchiumIV in ~ [15:46:32]
$ podman run -ith cont2 --name cont2 --rm --net net1 --net net2 arch-with-nc
[root@cont2 /]# ping cont1
PING cont1 (fde2:d4e8:4db8:c1d1::2) 56 data bytes
64 bytes from cont1 (fde2:d4e8:4db8:c1d1::2): icmp_seq=1 ttl=64 time=0.021 ms
^C
--- cont1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.021/0.021/0.021/0.000 ms
[root@cont2 /]# ping cont3
PING cont3 (fd61:6682:823a:14a::3) 56 data bytes
64 bytes from cont3 (fd61:6682:823a:14a::3): icmp_seq=1 ttl=64 time=0.027 ms
^C
--- cont3 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.027/0.027/0.027/0.000 ms
[root@cont2 /]# ping cont4
ping: cont4: Name or service not known
[root@cont2 /]#

~: podman

# mini_bomba @ miniarchiumIV in ~ [15:46:30]
$ podman run -ith cont3 --name cont3 --rm --net net2 --net net3 arch-with-nc
[root@cont3 /]# ping cont2
PING cont2 (fd61:6682:823a:14a::2) 56 data bytes
64 bytes from cont2 (fd61:6682:823a:14a::2): icmp_seq=1 ttl=64 time=0.051 ms
^C
--- cont2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.051/0.051/0.051/0.000 ms
[root@cont3 /]# ping cont4
PING cont4 (fdd7:697e:b1f6:8a28::2) 56 data bytes
64 bytes from cont4 (fdd7:697e:b1f6:8a28::2): icmp_seq=1 ttl=64 time=0.065 ms
64 bytes from cont4 (fdd7:697e:b1f6:8a28::2): icmp_seq=2 ttl=64 time=0.051 ms
^C
--- cont4 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1057ms
rtt min/avg/max/mdev = 0.051/0.058/0.065/0.007 ms
[root@cont3 /]# ping cont1
ping: cont1: Name or service not known
[root@cont3 /]#
```



Przykład



```
[root@cont1 /]# ping cont3
ping: cont3: Name or service not known
[root@cont1 /]# ping cont4
ping: cont4: Name or service not known
[root@cont1 /]#

# mini_bomba @ miniarchiumIV in ~ [15:49:11]
$ podman network connect cont1 net3
Error: no container with name or ID "net3" found: no such container

# mini_bomba @ miniarchiumIV in ~ [15:49:28] C:125
$ podman network connect net3 cont1

# mini_bomba @ miniarchiumIV in ~ [15:49:33]
$ podman attach cont1

[root@cont1 /]# ping cont4
PING cont4 (fdd7:697e:b1f6:8a28::2) 56 data bytes
64 bytes from cont4 (fdd7:697e:b1f6:8a28::2): icmp_seq=1 ttl=64 time=0.080 ms
^C
--- cont4 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.080/0.080/0.080/0.000 ms
[root@cont1 /]#

~: podman

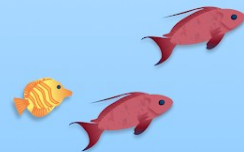
^C
--- cont1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.021/0.021/0.021/0.000 ms
[root@cont2 /]# ping cont3
PING cont3 (fd61:6682:823a:14a::3) 56 data bytes
64 bytes from cont3 (fd61:6682:823a:14a::3): icmp_seq=1 ttl=64 time=0.027 ms
^C
--- cont3 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.027/0.027/0.027/0.000 ms
[root@cont2 /]# ping cont4
ping: cont4: Name or service not known
[root@cont2 /]#

# mini_bomba @ miniarchiumIV in ~ [15:50:14]
$ podman network disconnect net2 cont2

# mini_bomba @ miniarchiumIV in ~ [15:50:26]
$ podman attach cont2

[root@cont2 /]# ping cont3
ping: cont3: Name or service not known
[root@cont2 /]#
```



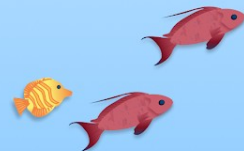


Rozdział 2

Docker Compose



Czym jest Docker Compose?



- Dokument YAML, który mówi jak stworzyć zestaw kontenerów
 - Opisuje kontenery, wraz z parametrami
 - Zmienne środowiskowe, wolumeny, wystawione porty, połączone sieci...
 - ... oraz inne obiekty, z których te kontenery korzystają
 - Czyli sieci i wolumeny
- Dokumentacja: docs.docker.com/reference/compose-file/



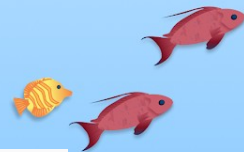
Dlaczego korzystać z Docker Compose?



- Ułatwia zarządzanie i modyfikację kontenerów
 - Nie trzeba pamiętać/wyciągać z historii shella komend na uruchomienie kontenera
 - Jeden plik na całą aplikację
 - Można plik wrzucić w gita



Przykład (wordpress)



- Definiuje dwa wolumeny
- Definiuje dwa kontenery
 - Główny serwis i bazę danych
 - Każdy kontener ma zmienne i wolumen
 - Główny serwis ma wystawiony port
- Ważne: mimo że nie zdefiniowano sieci, compose utworzy nową sieć domyślną dla tego „stosu usług”

```
services:

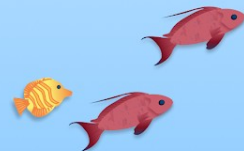
wordpress:
  image: wordpress
  restart: always
  ports:
    - 8080:80
  environment:
    WORDPRESS_DB_HOST: db
    WORDPRESS_DB_USER: exampleuser
    WORDPRESS_DB_PASSWORD: examplepass
    WORDPRESS_DB_NAME: exampledb
  volumes:
    - wordpress:/var/www/html

db:
  image: mysql:8.0
  restart: always
  environment:
    MYSQL_DATABASE: exampledb
    MYSQL_USER: exampleuser
    MYSQL_PASSWORD: examplepass
    MYSQL_RANDOM_ROOT_PASSWORD: '1'
  volumes:
    - db:/var/lib/mysql

volumes:
  wordpress:
  db:
```



Jak korzystać z Docker Compose?

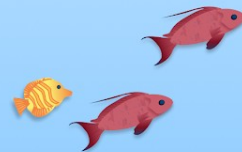


- Upewnij się, że masz zainstalowaną wtyczkę docker-compose
 - Działa ona również na najnowszym podmanie
- W katalogu z plikiem compose.yml:
 - docker compose up – postaw
 - docker compose down – wyłącz
 - + inne, rzadziej używane subkomendy

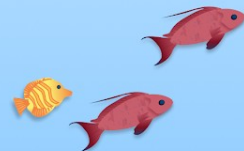


Rozdział 3

DNS



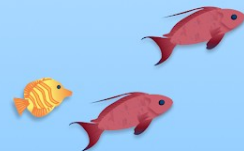
W jaki sposób działają domeny?



- Pakiety są adresowane na konkretne adresy IP
 - ... ale nikt nie wpisuje 142.250.109.136 zamiast youtube.com
- DNS (Domain Name System) – baza wpisów skojarzonych z domenami
 - Za pomocą odpowiednich rekordów można skojarzyć zarejestrowaną domenę z adresem IP



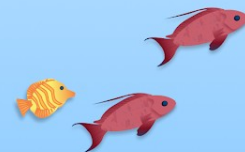
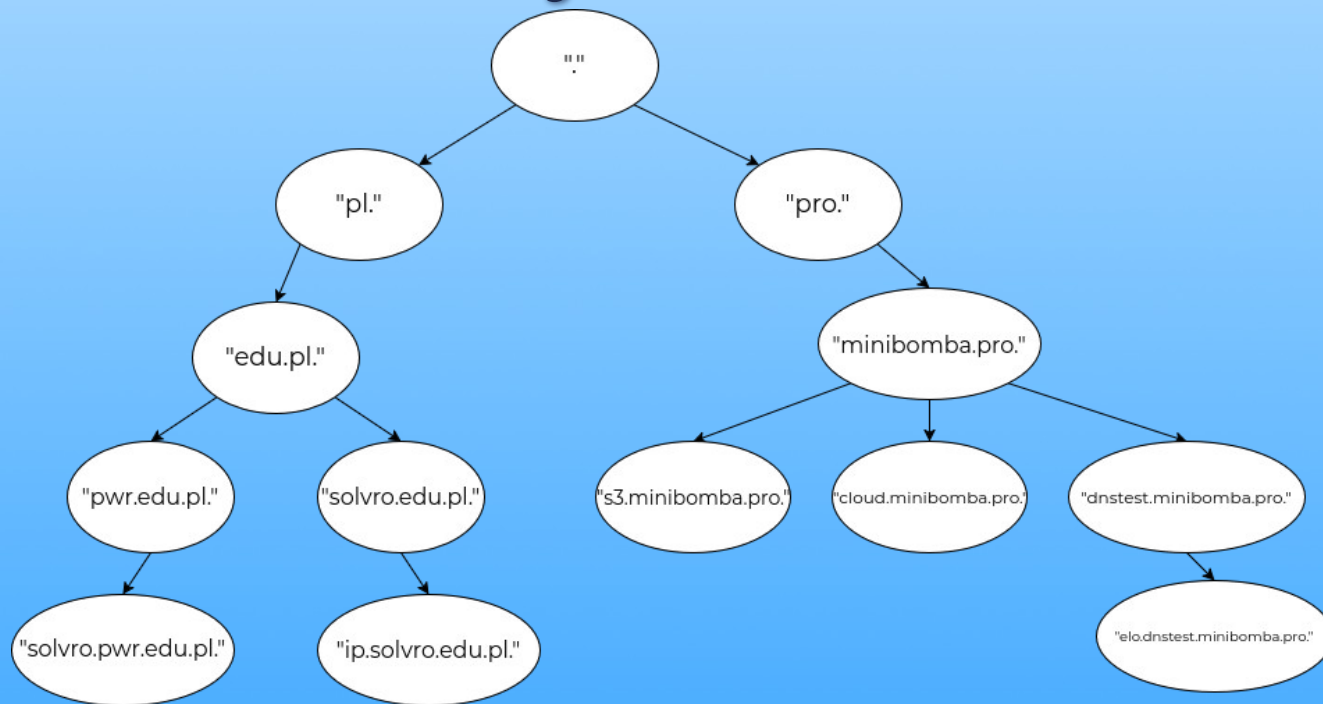
Typy serwerów DNS



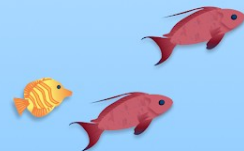
- Istnieje kilka typów serwerów DNS, ze względu na pełnione funkcje:
 - **Authoritative NS** – jest autorytatywnym źródłem informacji o wybranych „strefach”
 - ▶ Przykład: NSD, BIND 9
 - ▶ Rozróżniamy serwery główne (primary) i podrzędne (secondary)
 - Primary jest głównym źródłem informacji strefy
 - Secondary co jakiś czas synchronizuje swoją strefę z serwerem głównym
 - **Recursive resolver** – przyjmuje zapytania i kolejno (rekurencyjnie) odpytuje odpowiednie serwery autorytatywne
 - ▶ Przykład: Unbound, BIND 9
 - **Cache** – odpowiada na zapytania według zapamiętanych odpowiedzi, lub przekazuje je dalej
 - ▶ Często funkcja Recursive resolvera
 - **Stub** – po prostu przekazuje zapytania dalej
 - ▶ systemd-resolved w domyślnej konfiguracji jest serwerem typu stub



DNS jako drzewo



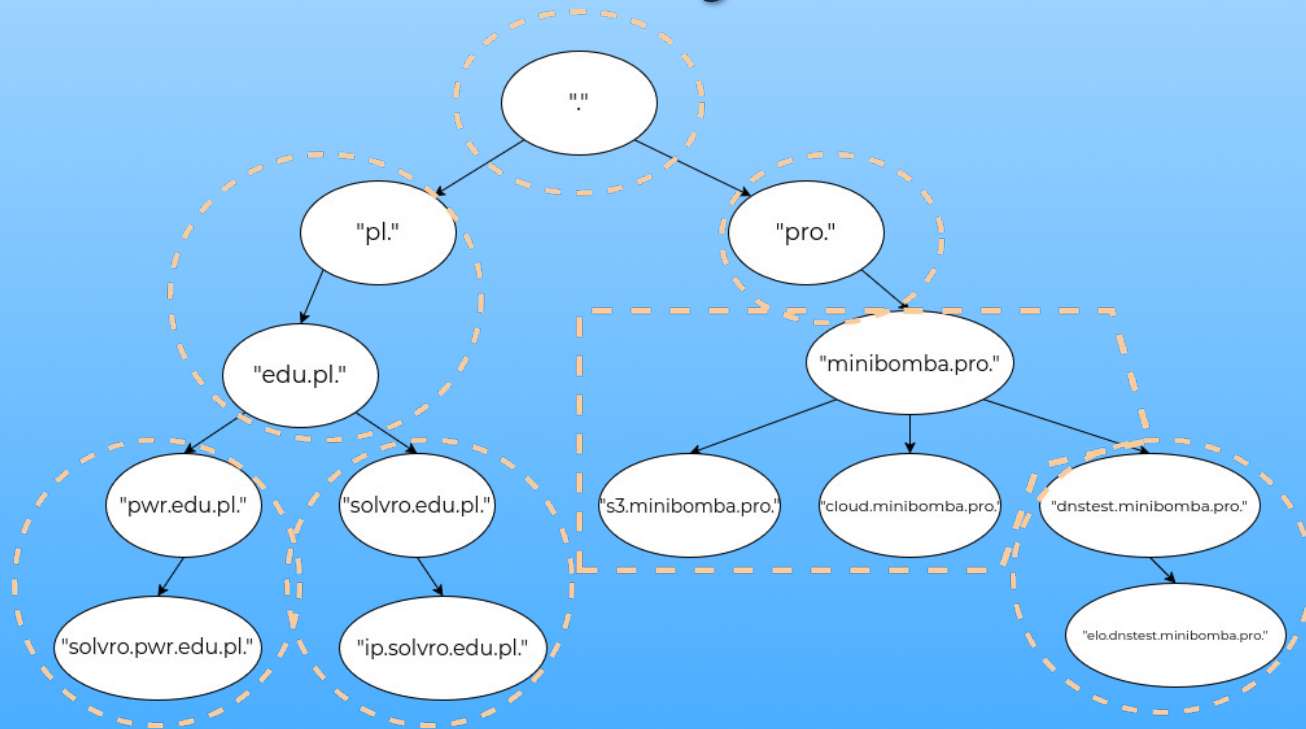
„Strefy”



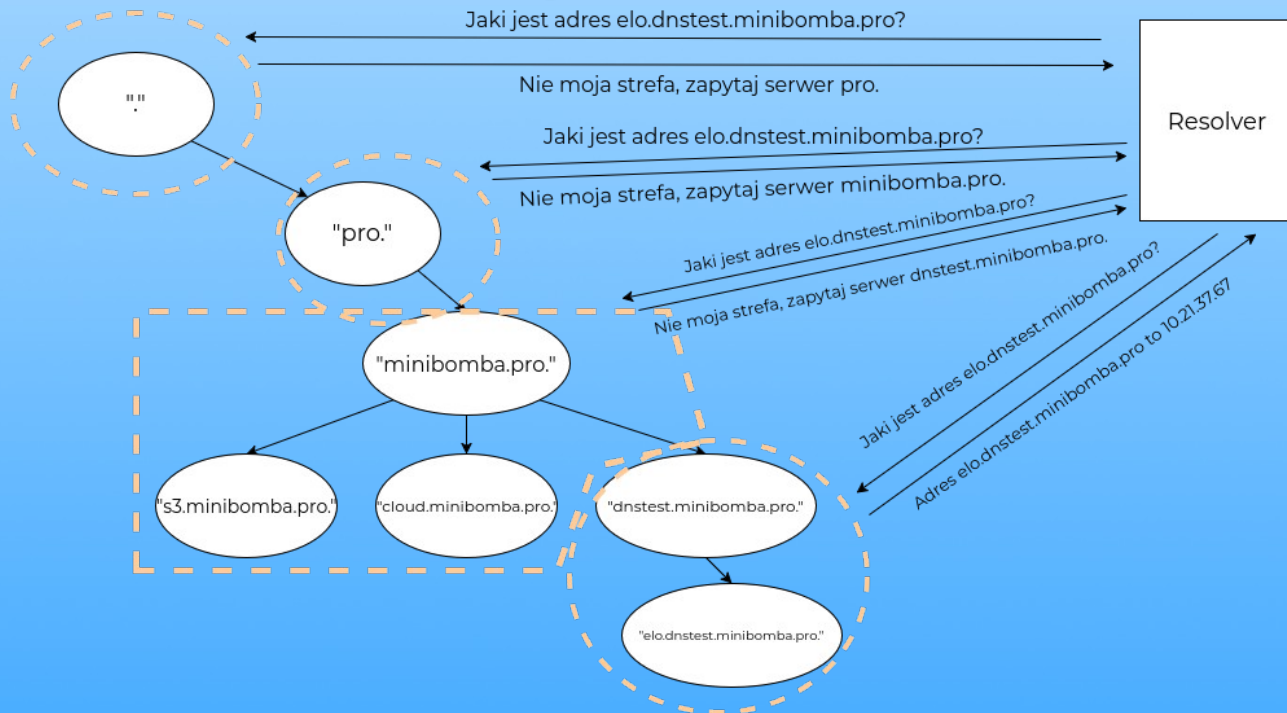
- Strefa to kawałek drzewa DNS, który ma jeden zestaw autorytatywnych serwerów DNS
- Wszystko zaczyna się od strefy „.”
- Strefa może odpowiednim rekordem oddelegować subdomenę innemu serwerowi (strefie)
 - Generalnie strefa „.” i wszystkie TLD delegują każdą subdomenę
 - Gdy rejestrujesz domenę, rejestr oddelegowuje ci kontrolę nad nią
- Punkty delegacji są punktami granicznymi stref



Wizualizacja stref



Rekurencyjne odpytywanie



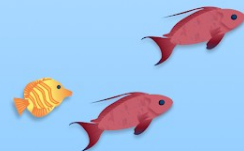
Rekurencyjne odpytywanie

```
~ : zsh — Konsola
Plik  Edycja  Widok  Zakładki  Wtyczki  Ustawienia  Pomoc
Nowa karta  Podziel widok  Zezwól na śledzenie myszy  Skopiuj  Wklej  Poszukaj

mini_bomba@miniarchiumIV ~$ drill -T elo.dnstest.minibomba.pro A
.      518400 IN      NS      l.root-servers.net.
.      518400 IN      NS      b.root-servers.net.
.      518400 IN      NS      a.root-servers.net.
.      518400 IN      NS      d.root-servers.net.
.      518400 IN      NS      h.root-servers.net.
.      518400 IN      NS      j.root-servers.net.
.      518400 IN      NS      k.root-servers.net.
.      518400 IN      NS      e.root-servers.net.
.      518400 IN      NS      i.root-servers.net.
.      518400 IN      NS      f.root-servers.net.
.      518400 IN      NS      g.root-servers.net.
.      518400 IN      NS      c.root-servers.net.
.      518400 IN      NS      m.root-servers.net.
pro.   172800 IN      NS      a0.pro.afiliast-nst.info.
pro.   172800 IN      NS      a2.pro.afiliast-nst.info.
pro.   172800 IN      NS      b0.pro.afiliast-nst.org.
pro.   172800 IN      NS      b2.pro.afiliast-nst.org.
pro.   172800 IN      NS      c0.pro.afiliast-nst.info.
pro.   172800 IN      NS      d0.pro.afiliast-nst.org.
minibomba.pro. 3600 IN      NS      dns111.ovh.net.
minibomba.pro. 3600 IN      NS      ns111.ovh.net.
dnstest.minibomba.pro. 3600 IN      NS      ns.dnstest.minibomba.pro.
dnstest.minibomba.pro. 3600 IN      NS      ns1.mschaet23.de.
elo.dnstest.minibomba.pro. 300 IN      A      10.21.37.67
dnstest.minibomba.pro. 300 IN      NS      ns.dnstest.minibomba.pro.
dnstest.minibomba.pro. 300 IN      NS      ns1.mschaet23.de.
mini_bomba@miniarchiumIV ~$
```



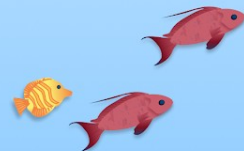
Rekordy



- **Rekordy w DNS są złożone z:**
 - Domeny, której dotyczą
 - TTL – okresu ważności rekordu
 - ▶ Dla serwerów cache, po upływie tego czasu muszą unieważnić zapamiętany wpis
 - Klasy, takie jak IN, CH, HS
 - ▶ Bo kiedyś myślano, że Internet nie będzie jedyną siecią sieci
 - Typu, definiującego typ i format zawieranych danych
 - Parametrów danego typu rekordu



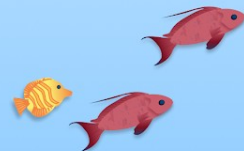
Ważne typy rekordów



- Praktycznie wszystkie ważne rekordy są klasy IN (internet)
 - Pewne zastosowanie mają też rekordy CH TXT
- **SOA – Start of Authority**
 - Pierwszy rekord każdej strefy, zawiera dane kontaktowe i konfigurację dla serwerów podrzędnych
- **NS – Name Server**
 - Informuje o serwerach obsługujących daną strefę
 - Powinien znaleźć się na początku każdej strefy; umieszczenie go na gałęzi strefy oznacza punkt delegacji
- **A/AAAA – Address**
 - Przypisuje domenie adres IP (A – v4; AAAA – v6)



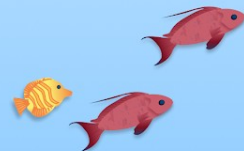
Ważne typy rekordów



- **CNAME – Canonical Name**
 - „ej, ta domena to tak naprawdę alias innej; ponów zapytanie dla tej drugiej domeny”
- **MX – Mail Exchange**
 - Informuje o nazwie serwera odpowiedzialnego za wymianę maili dla tej domeny
- **PTR – Pointer**
 - Używany przy zapytaniach Reverse DNS (w poddrzewie „in-addr.arpa” i „ip6.arpa”)
 - Przypisuje adresowi IP domenę



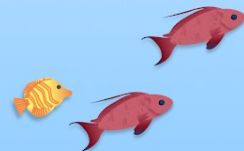
Ważne typy rekordów



- **SRV – Service**
 - Przypisuje wybranym serwisom domenę + niestandardowy port
- **HTTPS**
 - Zawiera domenę serwera odpowiedzialnego za stronę na tej domenie, wspierane jego protokoły alternatywnę i niestandardowy port
- **TXT**
 - Dowolny tekst, wykorzystywane do opisu polityk (np. SPF, DKIM, DMARC) lub do weryfikacji domen



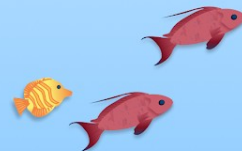
Rekordy „glue”



- Ważne: tylko rekordy A/AAAA zawierają adresy IP
 - HTTPS, SRV, MX, NS, ... wskazują na domeny, które powinny zawierać rekordy A/AAAA
- Co jeśli rekord NS wskazuje na serwer w domenie, którą on sam deleguje?
 - Strefa może zawierać niektóre rekordy dla domen, które nie są w niej zawarte
 - W takiej sytuacji strefa delegująca zawiera rekord A/AAAA dla domeny, na którą wskazuje rekord NS



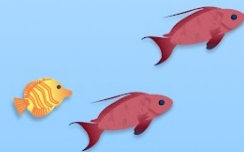
Przykłady stref



```
1 $TTL 3600
2 $ORIGIN minibomba.pro.
3
4 @                IN SOA   dns111.ovh.net. tech.ovh.net. (2086891651 86400 3600 3600000 60)
5 @                IN NS    dns111.ovh.net.
6 @                IN NS    ns111.ovh.net.
7
8 @                IN A      130.61.21.244
9 @                IN AAAA   2603:c020:8001:642::2137
10 @               IN TXT    "v=spf1 -all"
11 @               IN CAA    128 issue "letsencrypt.org"
12 @               IN CAA    128 issuewild "letsencrypt.org"
13
14 *                IN CNAME  minibomba.pro.
15 *.komputereg     IN CNAME  komputereg.minibomba.pro.
16 *.ovh1           IN CNAME  ovh1.minibomba.pro.
17 *.s3             IN CNAME  s3.minibomba.pro.
18 *.s3w           IN CNAME  s3.minibomba.pro.
19
20 ovh1             IN A      51.68.153.234
21 ovh1             IN AAAA   2001:41d0:602:1eea::1
22 ovh1             IN HTTPS  1 . alpn="h3,h2,http/1.1" port="443"
23 decrypt.ovh1    IN A      51.68.153.234
24 decrypt.ovh1    IN AAAA   2001:41d0:602:1eea::1
25
26 dnstest          IN NS     ns1.mschae23.de.
27 dnstest          IN NS     ns.dnstest.minibomba.pro.
28 ns.dnstest       IN A      51.68.153.234
29 ns.dnstest       IN AAAA   2001:41d0:602:1eea::1
30
31 _minecraft._tcp  IN SRV    0 5 25565 @
32 _minecraft._tcp.nomi-ceu IN SRV    0 5 25566 @
33 _minecraft._tcp.tfc IN SRV    0 5 25565 @
34
35 bazodanownik    IN A      10.22.37.1
36 cloud           IN CNAME  ovh1.minibomba.pro.
37 komputereg       IN A      10.22.37.10
38 komputereg       IN AAAA   2001:41d0:602:1eea:2137::10
39 resolver         IN CNAME  resolver.ovh1.minibomba.pro.
40 resolver.ovh1    IN A      51.68.153.234
41 resolver.ovh1    IN AAAA   2001:41d0:602:1eea::53
42 resolver.ovh1    IN HTTPS  1 . alpn="h3,h2,http/1.1" port="443"
43 s3              IN CNAME  ovh1.minibomba.pro.
44 s3w             IN CNAME  s3.minibomba.pro.
```

```
1 $ORIGIN dnstest.minibomba.pro.
2 $TTL 300
3
4 @ IN SOA (
5     ns.dnstest.minibomba.pro. ; primary server
6     minibomba.minibomba.pro. ; admin email
7     2026090501 ; serial
8     3600 ; refresh
9     300 ; retry
10    43200 ; expire
11    60 ; negative response ttl
12 )
13
14 ; the servers
15 @ IN NS ns.dnstest.minibomba.pro.
16 @ IN NS ns1.mschae23.de.
17
18 @ IN TXT "elo zelo!!!!"
19
20 ; the dns server
21 ns IN A      51.68.153.234
22 ns IN AAAA   2001:41d0:602:1eea::1
23
24 elo IN A 10.21.37.67
```



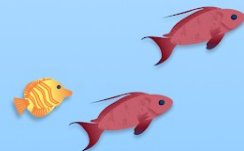


Rozdział 4

HTTPS



Jak skonfigurować HTTPS



- W poprzednim tygodniu pokazałem wam stawianie serwera HTTP
- Ale co, jeżeli chcielibyście przy swojej stronie mieć tą „kłódeczkę”?
- Potrzebujecie certyfikatu TLS...



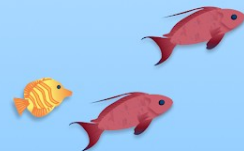
minibomba.pro



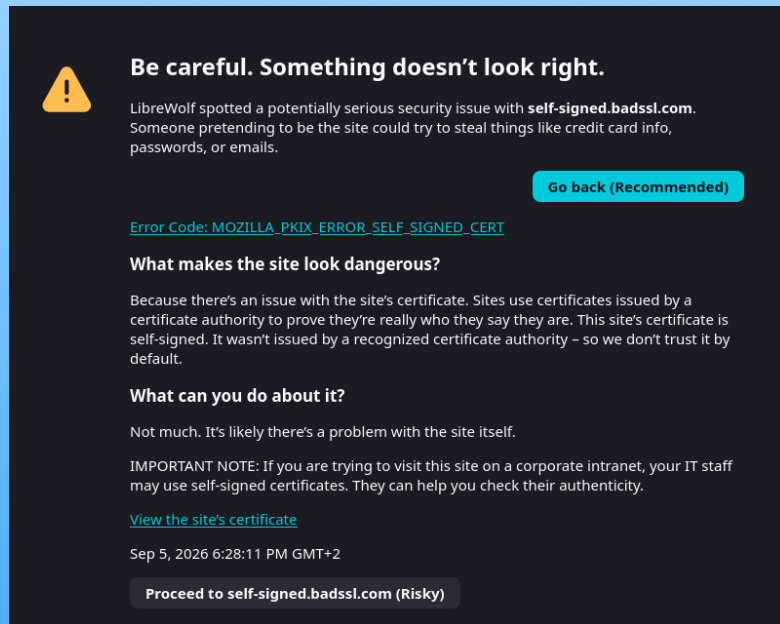
Connection secure >



Jak otrzymać certyfikat?



- W teorii certyfikat można wygenerować sobie samemu...
- ... ale przeglądarki nie ufają jakimś losowym certyfikatom



Be careful. Something doesn't look right.

LibreWolf spotted a potentially serious security issue with **self-signed.badssl.com**. Someone pretending to be the site could try to steal things like credit card info, passwords, or emails.

[Go back \(Recommended\)](#)

Error Code: MOZILLA_PKIX_ERROR_SELF_SIGNED_CERT

What makes the site look dangerous?

Because there's an issue with the site's certificate. Sites use certificates issued by a certificate authority to prove they're really who they say they are. This site's certificate is self-signed. It wasn't issued by a recognized certificate authority – so we don't trust it by default.

What can you do about it?

Not much. It's likely there's a problem with the site itself.

IMPORTANT NOTE: If you are trying to visit this site on a corporate intranet, your IT staff may use self-signed certificates. They can help you check their authenticity.

[View the site's certificate](#)

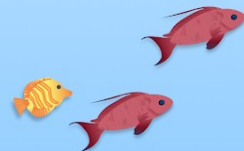
Sep 5, 2026 6:28:11 PM GMT+2

[Proceed to self-signed.badssl.com \(Risky\)](#)



download TLS certificate

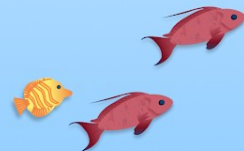
free crack no scam working 2016



- Przeglądarki ufają określonym CA – Urzędowi certyfikacji
- Wiele wymaga płatności za wydanie certyfikatu
 - Ale jako jakaś losowa strona na internecie nie potrzebujecie „certyfikatu premium”
- Urzędy takie jak Let's Encrypt i ZeroSSL wydają certyfikaty automatycznie i za darmo



Jak otrzymać certyfikat od LE



- Publiczne urzędy certyfikacji wydają certyfikaty tylko na publiczne domeny i adresy IP
- Automatyczne wydawanie bazuje na protokole ACME
 - Tworzymy konto w urzędzie
 - Zamawiamy certyfikat
 - określając metodę weryfikacji
 - Urząd sprawdza, czy faktycznie jesteśmy właścicielami domen/adresów IP
 - Często dając nam token do umieszczenia na stronie lub w DNSie
 - Urząd wydaje nam certyfikat

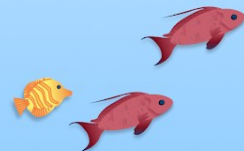


Nikt nie generuje certyfikatów ręcznie

- Do generowania certyfikatów służą narzędzia takie jak certbot lub acme.sh
 - Osobiście preferuję acme.sh
- Skryptom daje się listę domen do uwzględnienia w certyfikatach i dostęp do serwera HTTP lub DNS
- I ustawia się system operacyjny, by wywoływał te skrypty co jakiś czas – by odnawiały certyfikaty



Konfiguracja certyfikatów w nginx

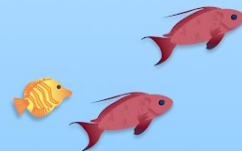


- Po wygenerowaniu certyfikatu, trzeba powiedzieć serwisom, by z niego korzystały
- W nginx służą do tego dyrektywy `ssl_*`

```
53     ssl_protocols TLSv1.3;  
54     ssl_prefer_server_ciphers on;  
55     ssl_certificate /etc/nginx/keys/minibomba.pro.crt;  
56     ssl_certificate_key /etc/nginx/keys/minibomba.pro.key;  
57     ssl_certificate_cache max=1000 inactive=20s valid=1m;  
58     ssl_session_cache shared:le_nginx_SSL:10m;  
59     ssl_session_timeout 1440m;  
60     ssl_session_tickets off;
```



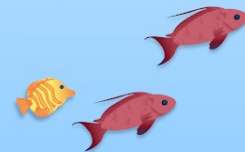
Własny urząd certyfikacji

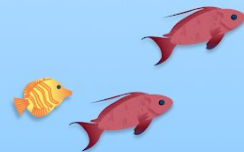


- ... ale po co? Przecież nikt temu nie będzie ufał...
 - Jeżeli chcecie zabezpieczyć ruch na sieci prywatnej...
 - ... lub z lokalnymi domenami
 - Albo może chcecie skorzystać z mTLS?
 - ▶ czyli wzajemnego uwierzytelniania serwera i klienta
 - ▶ Google wymusiło ostatnio na publicznych urządach przestanie wystawiania certyfikatów do uwierzytelniania klientów



Pytania?





**Dziękuję
za uwagę**

